

算法设计与分析·期中复习总结

考试时间：2026-04-15（后天）13:00-14:50，二教203+207，闭卷

⚠ 答题规范（必读！阅卷按此评分）

算法类型	必须包含的要素
递推方程	给出求解过程（迭代/递归树/主定理），写出结论
动态规划	算法思想 → 递推方程 → 边界条件 → 标记函数 → 自底向上 → 时间复杂度
贪心法	算法思想 → 贪心策略 → 正确性证明（归纳法 or 交换论证） → 复杂度
回溯/分支限界	解向量 → 搜索树类型 → 约束条件 → 代价函数（分支限界） → 复杂度
分摊分析	明确选用哪种方法 → 定义势函数/信用 → 计算每类操作的分摊代价 → 结论
线性规划	建模（目标函数+约束） → 求解过程 → 结果

一、数学基础

渐近记号

- $O(g)$: 上界, $\exists c, n_0: f(n) \leq c \cdot g(n)$
- $\Omega(g)$: 下界, $\exists c, n_0: f(n) \geq c \cdot g(n)$
- $\Theta(g)$: 紧界, 同时 O 和 Ω

常用求和公式

求和式	结果
$\sum_{i=1}^n i$	$\frac{n(n+1)}{2} = \Theta(n^2)$
$\sum_{i=1}^n i^2$	$\frac{n(n+1)(2n+1)}{6} = \Theta(n^3)$
$\sum_{i=0}^n x^i \ (x \neq 1)$	$\frac{x^{n+1}-1}{x-1}$; $x < 1$ 时 $\rightarrow \Theta(1)$; $x > 1$ 时 $\rightarrow \Theta(x^n)$
$\sum_{i=1}^n \frac{1}{i}$	$H_n = O(\log n)$ (调和级数)
$\sum_{i=1}^n \log i$	$O(n \log n)$
$\sum_{i=0}^{\infty} i \cdot x^i \ (x < 1)$	$\frac{x}{(1-x)^2}$

二、递推方程求解

方法1：迭代归纳法

直接迭代示例： $T(n) = T(n-1) + n, T(1) = 1$

$$T(n) = T(n-1) + n = T(n-2) + (n-1) + n = \cdots = T(1) + 2 + 3 + \cdots + n = \frac{n(n+1)}{2} - 1 = \Theta(n^2)$$

差消迭代示例：快速排序均值 $T(n) = \frac{2}{n} \sum_{k=1}^{n-1} T(k) + O(n)$

$$nT(n) = 2 \sum_{k=1}^{n-1} T(k) + O(n^2)$$

$$(n-1)T(n-1) = 2 \sum_{k=1}^{n-2} T(k) + O((n-1)^2)$$

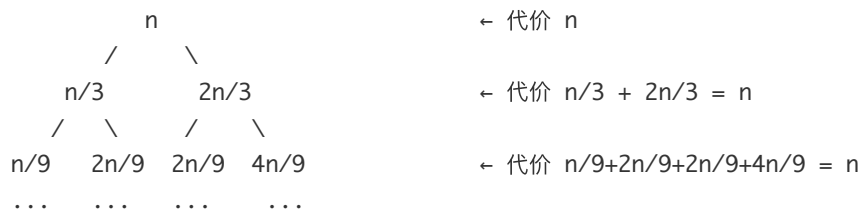
两式相减 $\rightarrow nT(n) - (n-1)T(n-1) = 2T(n-1) + O(n) \rightarrow T(n) = O(n \log n)$

换元迭代示例： $T(n) = T(\sqrt{n}) + 1$, 令 $m = \log n$, $S(m) = S(m/2) + 1 \rightarrow S(m) = \Theta(\log m) \rightarrow T(n) = \Theta(\log \log n)$

方法2：递归树法

例1: $T(n) = T(n/3) + T(2n/3) + n$

递归树（不均匀划分）：



每层代价均为 n

‡ 树高 = $\log_{3/2}(n)$
(最长路径：每次取 $2n/3$)

- 每层代价和为 n , 树高 $\log_{3/2} n$
- 结果: $T(n) = O(n \log n)$

例2: $T(n) = 2T(n/2) + n \log n$ (不能用主定理!)

递归树：

第0层：

n · log(n)

第1层：

(n/2)log(n/2) (n/2)log(n/2)

第2层：

... ...

第k层：

...

第logn层：

...

代价 = n · log(n)

代价 = n · (log n - 1)

代价 = n · (log n - 2)

代价 = n · (log n - k)

代价 = n · 0 = 0

总代价 = $\sum_{k=0}^{\log n} n \cdot (\log n - k)$

= n · $\sum_{j=0}^{\log n} j$

= n · log(n) · (log(n)+1)/2

= O(n · log²n)

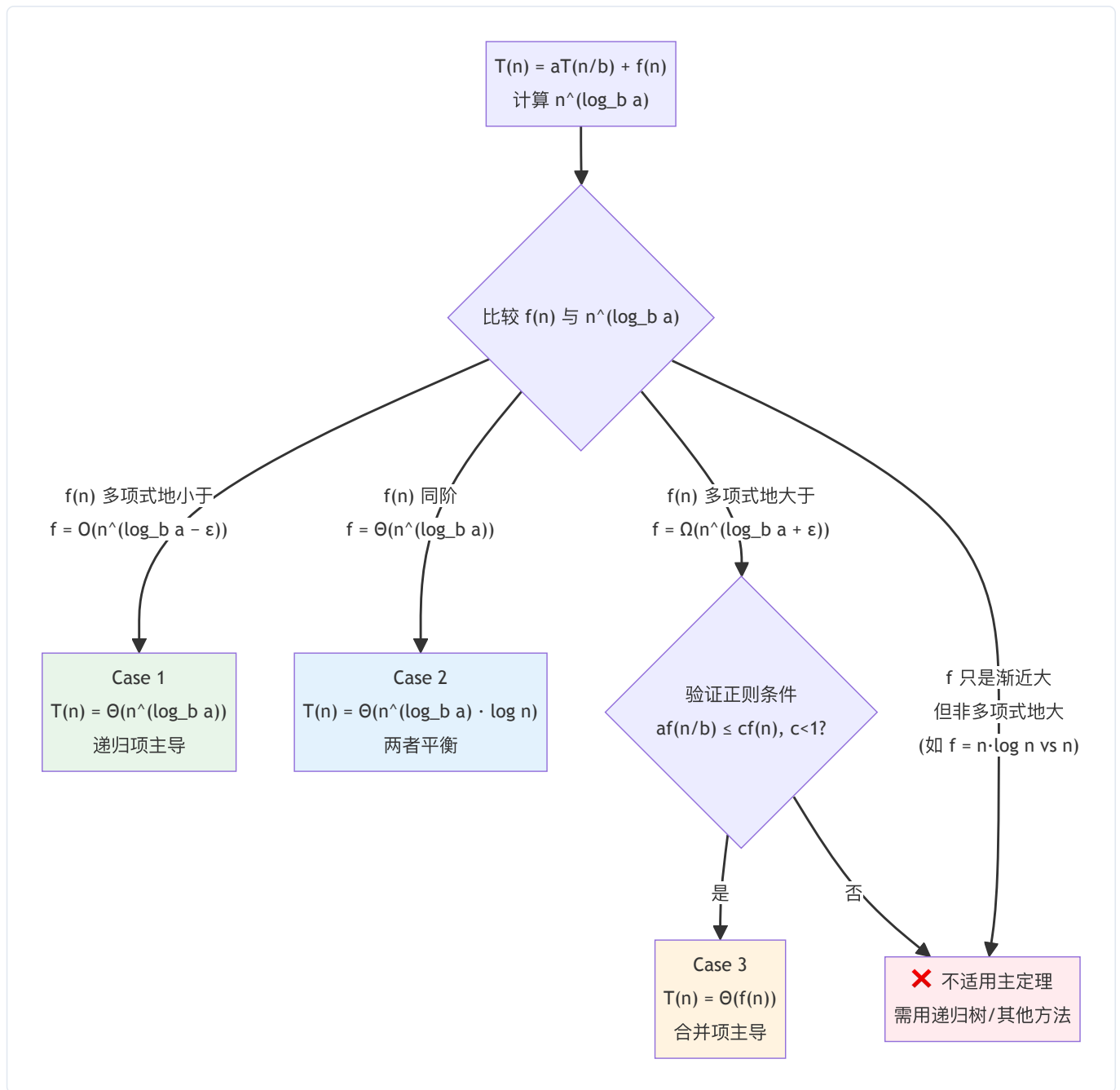
• $\sum_{k=0}^{\log n} n(\log n - k) = n \sum_{j=0}^{\log n} j = O(n \log^2 n)$

方法3：主定理（Master Theorem）

$T(n) = aT\left(\frac{n}{b}\right) + f(n), \quad a \geq 1, b > 1$

情形	条件	结论
Case 1	$f(n) = O(n^{\log_b a - \epsilon}), \epsilon > 0$	$T(n) = \Theta(n^{\log_b a})$
Case 2	$f(n) = \Theta(n^{\log_b a})$	$T(n) = \Theta(n^{\log_b a} \log n)$
Case 3	$f(n) = \Omega(n^{\log_b a + \epsilon})$ 且 $af(n/b) \leq cf(n) \ (c < 1)$	$T(n) = \Theta(f(n))$

主定理判断流程图：



典型应用：

递推方程	a, b	$n^{\log_b a}$	$f(n)$	情形	结论
$T(n) = 9T(n/3) + n$	9, 3	n^2	n	Case 1	$\Theta(n^2)$
$T(n) = T(2n/3) + 1$	1, 3/2	$n^0 = 1$	1	Case 2	$\Theta(\log n)$
$T(n) = 3T(n/4) + n \log n$	3, 4	$n^{0.79}$	$n \log n$	Case 3	$\Theta(n \log n)$
$T(n) = 4T(n/2) + n^2$	4, 2	n^2	n^2	Case 2	$\Theta(n^2 \log n)$
$T(n) = 2T(n/2) + n^2 \log n$	2, 2	n	$n^2 \log n$	Case 3	$\Theta(n^2 \log n)$
$T(n) = 2T(n/2) + n \log n$	2, 2	n	$n \log n$	不适用	$O(n \log^2 n)$

⚠️ $T(n) = 2T(n/2) + n \log n$ 不适用主定理： $n \log n$ 不是 $\Omega(n^{1+\epsilon})$ ，需用递归树。

方法4：尝试法（处理 floor/ceiling）

$T(n) = 2T(\lfloor n/2 \rfloor) + n$, $T(1) = 1$, 猜 $T(n) \leq cn \log n$:

- 假设 $T(\lfloor n/2 \rfloor) \leq c \lfloor n/2 \rfloor \log \lfloor n/2 \rfloor$
- $T(n) \leq 2 \cdot c \cdot \frac{n}{2} \log \frac{n}{2} + n = cn(\log n - 1) + n = cn \log n - cn + n \leq cn \log n \quad (c \geq 1)$
- 验证边界: $T(2) = 4 \leq 2 \cdot 2 \cdot \log 2 = 4$, 取 $c = 2 \checkmark$

三、分治策略

基本思想

均衡划分 → 递归求解子问题 → 合并结果

经典算法复杂度

算法	递推方程	结论
归并排序	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$
二分查找	$T(n) = T(n/2) + O(1)$	$O(\log n)$
Karatsuba 整数乘法	$W(n) = 3W(n/2) + cn$	$O(n^{\log_2 3}) \approx O(n^{1.59})$
Strassen 矩阵乘法	$W(n) = 7W(n/2) + 18(n/2)^2$	$O(n^{\log_2 7}) \approx O(n^{2.807})$
最大子段和(分治)	$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$
快速排序(最坏)	$W(n) = W(n-1) + O(n)$	$\Theta(n^2)$
快速排序(最好/均衡)	$T(n) = 2T(n/2) + O(n)$	$\Theta(n \log n)$
快速排序(均值)	差消法	$O(n \log n)$
最近点对(预处理)	$W(n) = T(n) + O(n \log n)$, $T(n) = 2T(n/2) + O(n)$	$O(n \log n)$
FindMaxMin	两两一组	$W(n) = \lceil 3n/2 \rceil - 2$
找第二大(锦标赛)	—	$W(n) = n + \lceil \log n \rceil - 2$

快排 Partition 过程示例 (pivot = A[p] = 27):

27 99 0 8 13 64 86 16 7 10 88 25 90
 $p=x$ $j \leftarrow i \leftarrow$

第1轮: j 找到25 $\leq$ 27, i 找到99 $>$ 27 $\rightarrow$ 交换

27 [25] 0 8 13 64 86 16 7 10 88 [99] 90

第2轮: j 找到10, i 找到64 $\rightarrow$ 交换

27 25 0 8 13 [10] 86 16 7 [64] 88 99 90

第3轮: j 找到7, i 找到86 $\rightarrow$ 交换

27 25 0 8 13 10 [7] 16 [86] 64 88 99 90

第4轮: $j=16$ 处, i 越过 $j \rightarrow i>j$, 结束

交换 $A[p]$ 与 $A[j]$: 16 25 0 8 13 10 7 [27] 86 64 88 99 90

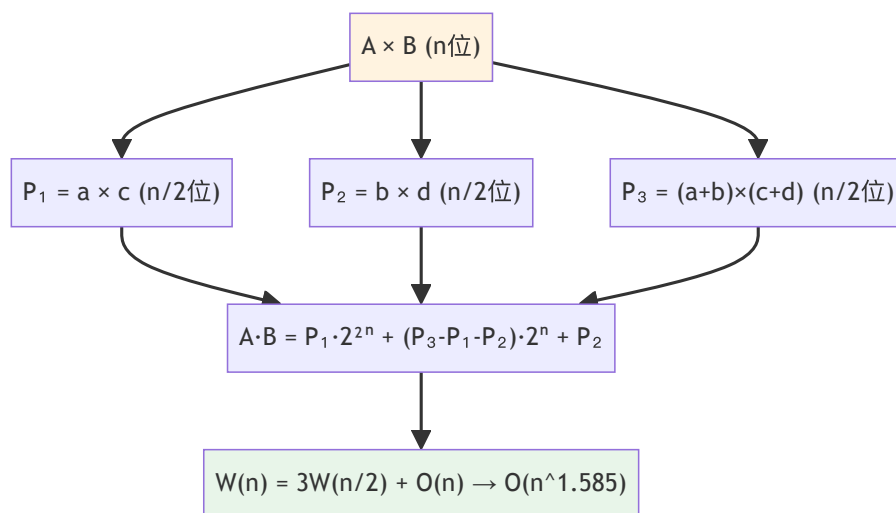
$\leftarrow$ 全部 $\leq$ 27 $\rightarrow$ pivot $\leftarrow$ 全部 $\geq$ 27 $\rightarrow$

Karatsuba 关键思想

令 $A = a \cdot 2^n + b$, $B = c \cdot 2^n + d$, 则:

$$A \cdot B = ac \cdot 2^{2n} + ((a+b)(c+d) - ac - bd) \cdot 2^n + bd$$

只需 3 次乘法: ac 、 bd 、 $(a+b)(c+d)$



Strassen 矩阵乘法: 类似思想

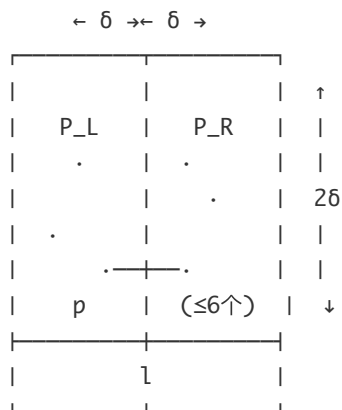
普通矩阵乘法: 8 次子矩阵乘法 $\rightarrow W(n)=8W(n/2)+O(n^2) \rightarrow O(n^3)$

Strassen: 7 次子矩阵乘法 $\rightarrow W(n)=7W(n/2)+O(n^2) \rightarrow O(n^{2.807})$

$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$
 减少 1 次乘法
 8次 $\rightarrow$ 7次
 代价: 更多加法 $O(n^2)$
 但乘法主导复杂度

最近点对关键分析

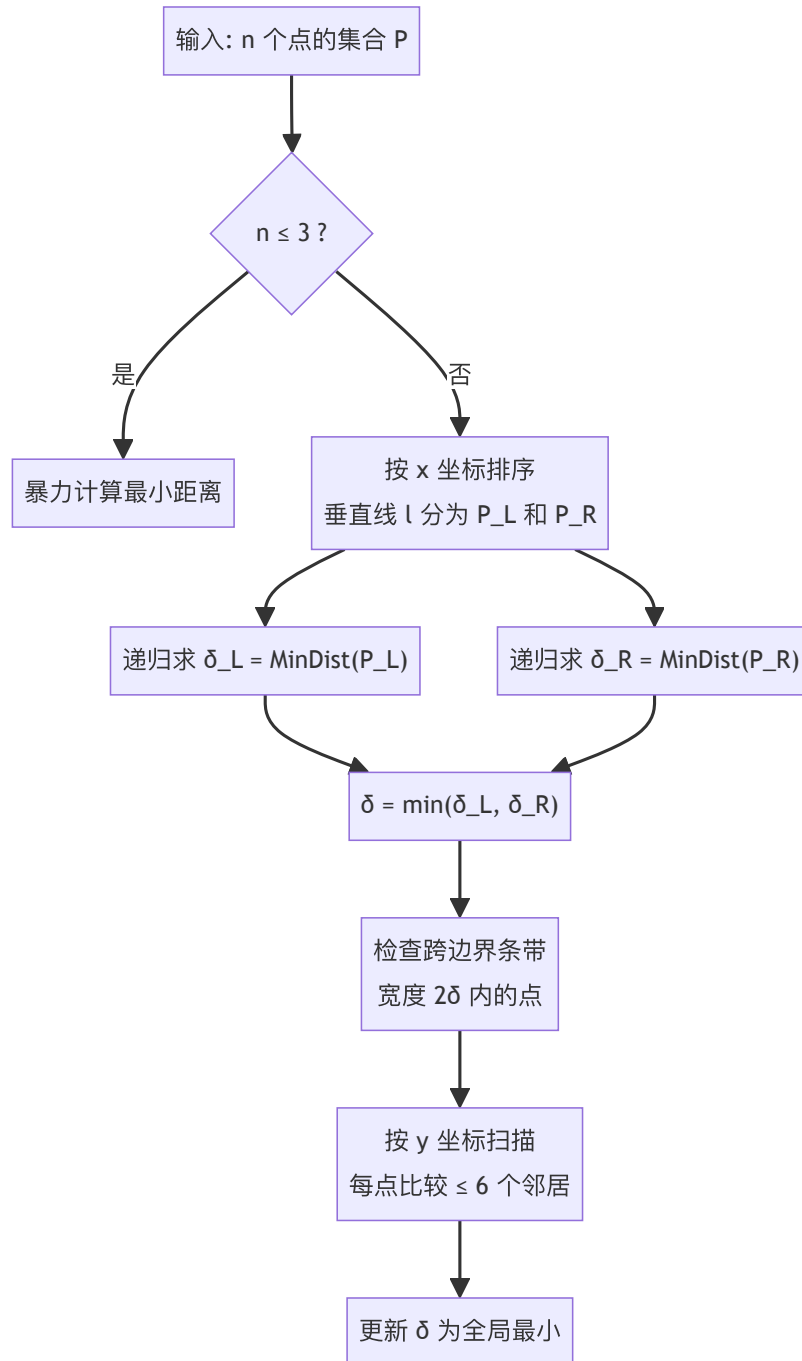
最近点对 - 跨边界条带示意图：



对于 p 点，对面 $\delta \times 2\delta$ 区域划分为 $2 \times 3 = 6$ 个小格
 每格最多 1 个点（否则同侧距离 $< \delta$ 矛盾）
 → 每个点只需比较 ≤ 6 个对面点 → 合并 $O(n)$

- 分治划分，两侧最小距离 $\delta = \min(\delta_L, \delta_R)$
- 中间条带 2δ 宽内，每个点最多与对面 6 个点比较
- 预排序改进： $W(n) = T(n) + O(n \log n)$ ，其中 $T(n) = 2T(n/2) + O(n) \rightarrow T(n) = O(n \log n)$ ，总 $W(n) = O(n \log n)$

最近点对算法流程图：



分治改进两途径

1. **代数变换**: 减少子问题数目 (如 Karatsuba: $4 \rightarrow 3$, Strassen: $8 \rightarrow 7$)
2. **预处理**: 先对数据排序, 降低每层合并代价

四、动态规划

基本要素 (答题必写)

1. **优化原则**: 最优解的子结构也是最优解
2. **递推方程** + 边界条件

3. 标记函数 (记录选择路径)

4. 自底向上填表

5. 时间/空间复杂度

矩阵链乘

$m[i, j]$: 矩阵 $A_i \cdots A_j$ 的最优计算代价

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + P_{i-1} \cdot P_k \cdot P_j\} & i < j \end{cases}$$

- 时间复杂度: $O(n^3)$, 空间 $O(n^2)$
- 按对角线 $l = 2, 3, \dots, n$ 顺序计算

矩阵链乘填表顺序 (n=4, 按对角线从短到长):

	j=1	j=2	j=3	j=4
i=1	[0]	[l=2①]	[l=3③]	[l=4⑥]
i=2		[0]	[l=2②]	[l=3⑤]
i=3			[0]	[l=2④]
i=4				[0]

填表顺序: 对角线 $l=1$ (主对角线=0)

- $l=2$ ($m[1, 2]$, $m[2, 3]$, $m[3, 4]$)
- $l=3$ ($m[1, 3]$, $m[2, 4]$)
- $l=4$ ($m[1, 4]$ ← 最终答案)

LCS (最长公共子序列)

$$C[i, j] = \begin{cases} 0 & i = 0 \text{ 或 } j = 0 \\ C[i - 1, j - 1] + 1 & x_i = y_j \\ \max(C[i, j - 1], C[i - 1, j]) & x_i \neq y_j \end{cases}$$

标记函数 $B[i, j] \in \{\nwarrow, \leftarrow, \uparrow\}$, 回溯构造解: $O(m + n)$

时间复杂度: $O(mn)$, 空间 $O(mn)$ 或 $O(\min(m, n))$

LCS 填表示例: X = "ABCB", Y = "BDCAB"

	∅	B	D	C	A	B
∅	[0	0	0	0	0	0]
A	[0	0	0	0	↖1	1]
B	[0	↖1	1	1	1	↖2]
C	[0	1	1	↖2	2	2]
B	[0	↖1	1	2	2	↖3]

↖ = 匹配(x_i=y_j), ← = 取左, ↑ = 取上
回溯路径(粗体↖): B[4,5]→B[3,3]→B[1,4] → LCS = "BCB"

最大子段和（三种算法）

方法	复杂度
暴力枚举	$O(n^3)$
分治	$O(n \log n)$
动态规划	$O(n)$

动态规划递推:

$$C[i] = \max\{C[i - 1] + A[i], A[i]\}, \quad C[0] = 0$$

$$OPT = \max_{0 \leq i \leq n} \{C[i]\}$$

最大子段和 DP 过程示例: A = [-2, 11, -4, 13, -5, 2]

i:	0	1	2	3	4	5	6
A[i]:		-2	11	-4	13	-5	2
C[i]:	0	-2	11	7	20	15	17
		↑	↑		↑		
		重新开始	max{-2+11, 11}		max{7+13, 13}		
		max{0+(-2), -2}	=11		=20 ← OPT!		
		=-2					

OPT = max(0, -2, 11, 7, 20, 15, 17) = 20
对应子段: A[2..4] = [11, -4, 13]

投资问题

$$F_k(x) = \max_{0 \leq x_k \leq x} \{f_k(x_k) + F_{k-1}(x - x_k)\}$$

时间复杂度: $O(nm^2)$ (n 个项目, m 单位资金, 每步 $O(m)$, 共 n 步)

面试分派 (2023年真题原型)

$dp[i][j]$: 前 i 人中有 j 人去 A 市的最小费用

$$dp[i][j] = \min(dp[i-1][j-1] + \text{cost}[i][0], dp[i-1][j] + \text{cost}[i][1])$$

$$1 \leq i \leq 2N, \quad 1 \leq j \leq \min(i, N)$$

$$dp[0][0] = 0, \quad dp[i][0] = \sum_{k=1}^i \text{cost}[k][1]$$

时间复杂度: $O(N^2)$

五、贪心法

基本思想

每步做局部最优选择, 证明局部最优导致全局最优。

证明方法 (考试必须给出之一)

1. 归纳法: 对算法步骤或问题规模归纳, 证明贪心解不差于任何解
2. 交换论证: 假设最优解与贪心解不同, 通过交换操作证明可改善至贪心解而不变差

活动选择问题

策略: 按结束时间 f_i 升序排序, 每次选结束时间最早的相容活动。

活动选择示例 (按 f_i 排序后):

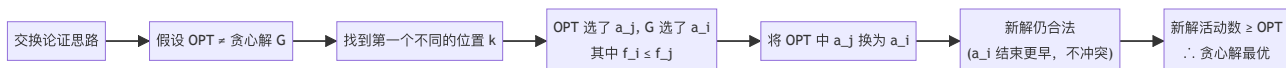
时间轴 → 0 1 2 3 4 5 6 7 8 9 10 11 12

a1:	■■	$f_1=1$ ✓选
a2:	■■■■	$f_2=3$
a3:	■■■■■	$f_3=4$ ✓选
a4:	■■■■	$f_4=5$
a5:	■■■■■■■	$f_5=7$
a6:	■■■■■■■■	$f_6=8$ ✓选
a7:	■■■■■■■■■	$f_7=10$
a8:	■■■■	$f_8=11$ ✓选

贪心选择: $a_1 \rightarrow a_3 \rightarrow a_6 \rightarrow a_8$ (共4个)

每次选结束最早且与已选不冲突的活动

正确性 (交换论证): 设 OPT 第一个选的不是结束最早的 a_1 , 将 OPT 中第一个活动换成 a_1 , 不减少相容活动数量 (a_1 结束更早, 不会挤占后面的空间)。



最优装载

策略: 按重量升序排序, 依次装入直到超重。

证明 (归纳法): 设算法选了 x_i , 任何最优解若不选 x_i 则可交换使得不更差。

最小延迟调度

策略: 按截止时间 d_i 升序排序。

证明 (交换论证): 若有"逆序" (i 在 j 后但 $d_i < d_j$), 交换两者不增加最大延迟。

面试问题 (2023) 贪心解法

策略: 按 $\text{cost}[i][0] - \text{cost}[i][1]$ 从大到小排序, 前 N 人去 B 市, 其余去 A 市。

证明 (交换论证): 若最优解中有 i 去 A 市、 j 去 B 市, 且 $\text{cost}[i][0] - \text{cost}[i][1] > \text{cost}[j][0] - \text{cost}[j][1]$, 则将 i, j 互换可降低总费用, 矛盾。

六、回溯与分支限界

搜索空间类型

问题	搜索树类型	叶节点数
n 皇后	n 叉树	—
0-1 背包	子集树 (二叉)	2^n
货郎问题(TSP)	排列树	$(n-1)!$
分派问题	排列树	$n!$

子集树 vs 排列树：

子集树 (0-1背包, $n=3$):

root

/ \

$x_1=1$ $x_1=0$

/ \ / \

$x_2=1$ $x_2=0$ $x_2=1$ $x_2=0$

/ \ / \ / \ / \

1 0 1 0 1 0 1 0

叶子数: $2^n = 8$

每层选"要/不要"

排列树 (TSP/分派, $n=3$):

root

/ | \

1 2 3

/ \ / \ / \

2 3 1 3 1 2

| | | | |

3 2 3 1 2 1

叶子数: $n! = 6$

每层选"下一个是谁"

4皇后回溯搜索树示例 (剪枝后):

root

/ | | \

$x_1=1$ $x_1=2$ $x_1=3$ $x_1=4$

/ | \ / | \

$x_2=2$ 3 4 1 3 4

$\times$ | | $\times$ | |

x_3 x_3 x_3 x_3

=2 =2 =1 =1

$\times$ | | $\times$

x_4 x_4

=3 =4

$\times$ ✓ 解:(2,4,1,3)

$\times$ = 剪枝 (同列/对角线冲突)

✓ = 找到一个可行解

多米诺性质

如果约束函数不满足 $P(x_1, \dots, x_{k+1})$, 则 $P(x_1, \dots, x_k)$ 也不满足 (向上传播剪枝)。

算法设计要素 (答题必须包含)

- 解向量 $X = (x_1, x_2, \dots, x_n)$, 每个 x_i 的取值范围
- 分支约束条件: 节点可行性约束
- 搜索策略: 深度优先 (回溯) or 最小代价优先 (分支限界)
- 存储路径: 记录最优解的路径

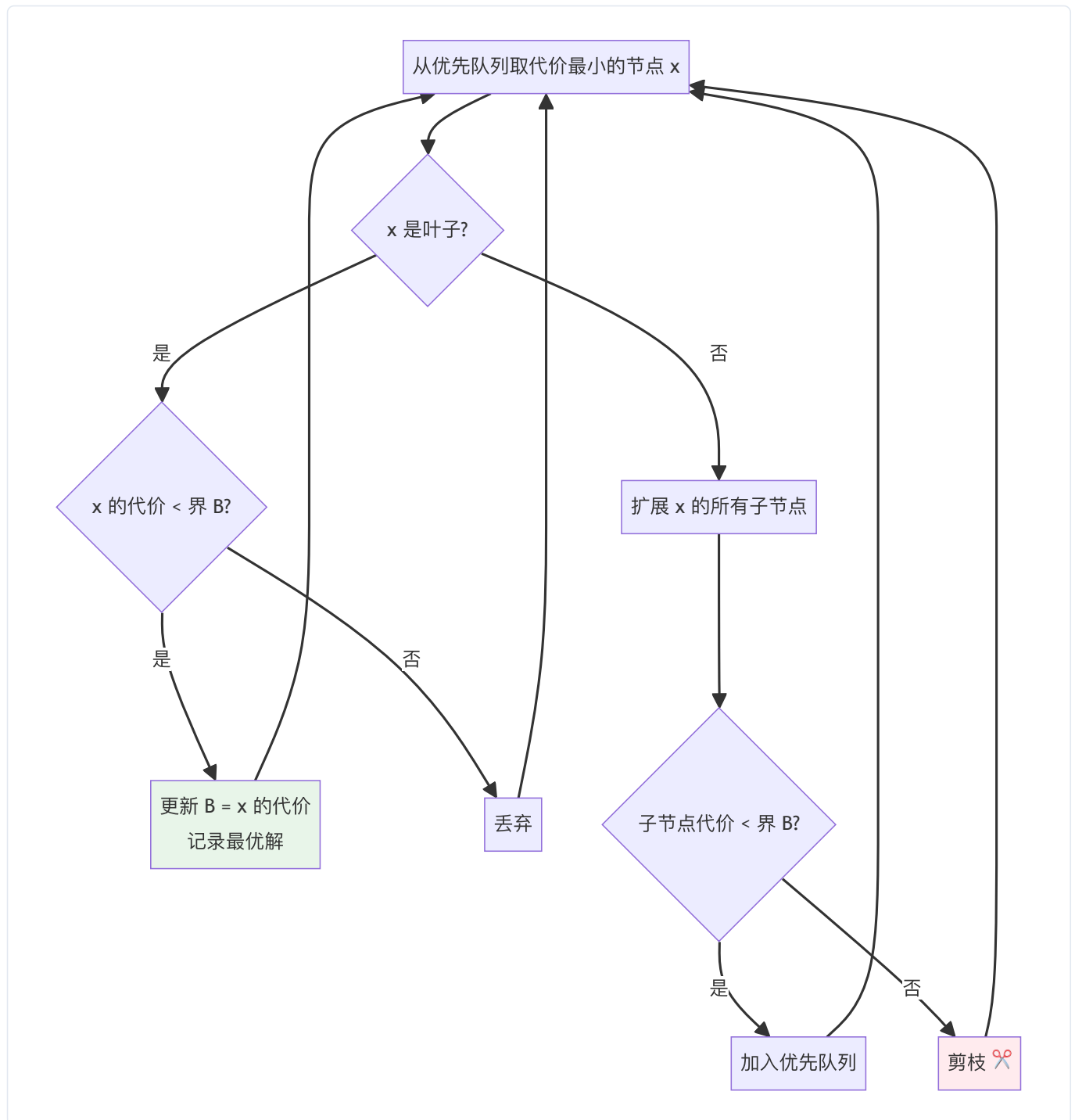
效率估计

$W(n) = p(n) \cdot f(n)$, 其中 $p(n)$ 是"有效节点"数, $f(n)$ 是每节点处理代价。

蒙特卡洛估计: 随机采样估计 $p(n)$ 。

分支限界关键

- 代价函数: 当前路径代价 + 子树中可能解的乐观估计 (下界)
- 界 B : 当前找到的最好可行解代价
- 剪枝条件: 代价函数值 $> B$ 时剪枝 (最小化问题)



分派问题代价函数 (2023真题)

设已分派 $x_1, \dots, x_k$, 代价函数:

$$F(x_1, \dots, x_k) = \sum_{i=1}^k C(i, x_i) + \sum_{i=k+1}^n \min\{C(i, t) \mid t \notin \{x_1, \dots, x_k\}\}$$

含义：已确定的代价 + 未分派人员各取最小代价的下界。

七、线性规划

基本建模

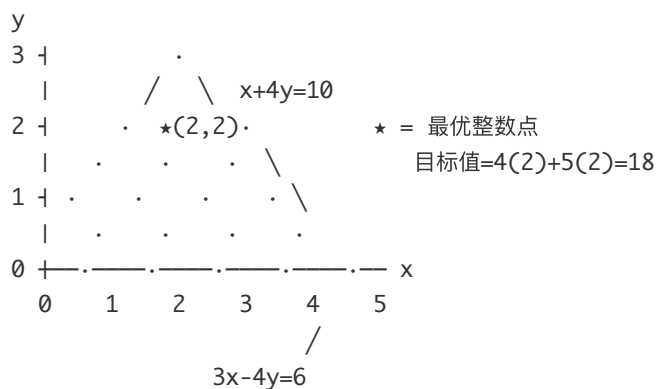
$$\max c^T x \quad \text{s.t. } Ax \leq b, x \geq 0$$

整数线性规划 (ILP) 求解

1. 图解法：画出可行域，找整数点最大化目标函数
2. 分支限界法：先求 LP 松弛，对非整数变量分支
3. 枚举法：枚举所有可行整数解

2023 典型题： $\max 4x + 5y$, s.t. $x + 4y \leq 10$, $3x - 4y \leq 6$, $x, y \geq 0$, $x, y \in \mathbb{Z}$

ILP 图解法：



可行域：两条线与坐标轴围成的三角区域

目标函数 $4x + 5y$ 等值线斜率 = $-4/5$

移动等值线直到最后接触可行域内的整数点 $\rightarrow (2, 2)$

- 最优解： $x = 2, y = 2$, 目标值 = $4 \cdot 2 + 5 \cdot 2 = 18$

八、平摊分析

三种方法对比

方法	核心思想	关键要求
聚集分析	T_{total}/n	直接计算总代价
记账法	为操作预付"信用", 存在对象上备用	任意时刻总信用 ≥ 0
势能法	定义势函数 Φ , $\hat{a}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$	$\Phi(D_n) \geq \Phi(D_0)$ 则 $\sum \hat{a}_i$ 是 $\sum c_i$ 上界

栈操作 (PUSH / POP / MULTIPOP)

记账法直观图示 (栈):

PUSH(a): 付 2 元

| a | ←存1元

| |

实际代价1
+存款1=2

PUSH(b): 付 2 元

| b | ←存1元

| a | ←仍有1元

实际代价1
+存款1=2

MULTIPOP(2): 付 0 元

| | b的1元付POP(b)

| | a的1元付POP(a)

实际代价2(两次弹出)
用存款抵付→0

聚集分析: 每个对象最多被 PUSH 1 次 POP 1 次, n 次操作总代价 $O(n)$, 平摊代价 $O(1)$ 。

记账法: PUSH 收费 2 (1 实际 + 1 存在对象上), POP 收费 0 (用存款), MULTIPOP 收费 0。

势能法: Φ = 栈中对象数

操作	实际代价 c_i	$\Delta\Phi$	平摊代价 $\hat{a}_i$
PUSH	1	+1	2
POP	1	-1	0
MULTIPOP(k)	$\min(s, k)$	$-\min(s, k)$	0

二进制计数器 (INCREMENT)

INCREMENT 聚集分析 - 翻转次数统计:

计数值: 0 1 2 3 4 5 6 7 8

二进制: 000 001 010 011 100 101 110 111 1000

A[0]: 0→1 1→0 0→1 1→0 0→1 1→0 0→1 1→0 每次都翻

A[1]: 0→1 1→0 0→1 1→0 0→1 1→0 0→1 1→0 每2次翻1次

A[2]: 0→1 1→0 0→1 1→0 0→1 1→0 0→1 1→0 每4次翻1次

A[3]: 0→1 1→0 0→1 1→0 0→1 1→0 0→1 1→0 每8次翻1次

翻转数: 1 2 1 3 1 2 1 4 ...

n次操作总翻转 $\leq n + n/2 + n/4 + \dots < 2n \rightarrow$ 平摊 $O(1)$

聚集分析： n 次 INCREMENT 总翻转次数 $< 2n$ (第 k 位每 2^k 次操作翻转 1 次)，平摊代价 $O(1)$ 。

记账法：每次 INCREMENT 收费 2 (1 翻 $0 \rightarrow 1$, 1 存在该位上备用)，平摊代价 = 2。

势能法： $\Phi = A$ 中 1 的个数

- 设第 i 次操作翻转 t_i 个 1 为 0，实际代价 $c_i = t_i + 1$
- 操作后 1 的个数 $b_i \leq b_{i-1} - t_i + 1$ (最多新增 1 个 1)
- 势差 $\Phi(D_i) - \Phi(D_{i-1}) \leq 1 - t_i$
- 平摊代价 $\hat{a}_i \leq (t_i + 1) + (1 - t_i) = 2$

初值非零：设初始有 b_0 个 1, n 次操作后有 b_n 个 1, 则：

$$\sum c_i \leq \sum \hat{a}_i + \Phi(D_0) - \Phi(D_n) \leq 2n - b_n + b_0 = O(n)$$

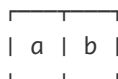
动态表（插入操作）

动态表扩张过程：

插入 1: size=1, num=1



插入 2: 满! 扩张→size=2



代价=2(复制a+插入b)

插入 3: 满! 扩张→size=4



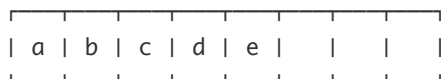
插入 4:



代价=1

代价=3(复制a,b+插入c)

插入 5: 满! 扩张→size=8



代价=5(复制a,b,c,d+插入e)

实际代价序列: 1, 2, 3, 1, 5, 1, 1, 1, ...

尖峰出现在 $i=2^k+1$

总代价 $\leq 3n \rightarrow$ 平摊=3

势函数： $\Phi(T) = 2 \cdot \text{num}[T] - \text{size}[T]$

不触发扩张：

$$\hat{a}_i = 1 + (2 \cdot \text{num}_i - \text{size}_i) - (2 \cdot \text{num}_{i-1} - \text{size}_{i-1}) = 3$$

触发扩张 ($\text{num}_i = \text{size}_{i-1} + 1$, $\text{size}_i = 2 \cdot \text{size}_{i-1}$):

$$\hat{a}_i = \text{num}_i + 2 - (\text{num}_i - 1) = 3$$

两种情况平摊代价均为 $3 = O(1)$ 。

含删除（满时扩张，1/4 时收缩）： $\Phi(T) = \max(2 \cdot \text{num}[T] - \text{size}[T], \text{size}[T]/2 - \text{num}[T])$

- 插入平摊代价 ≤ 3 ，删除平摊代价 ≤ 2

最小队列（2023真题）

用单调辅助队列维护最小值，势函数 $\Phi(Q) = \text{辅助队列长度}$

操作	实际代价	$\Delta\Phi$	平摊代价
入队	$1 + k$ （删 k 个+插 1）	$-k + 1$	2
出队	1（可能额外出辅助队头）	≤ 0	≤ 1
求最小值	1	0	1

每种操作均摊代价 $O(1)$ 。

九、历年考情分析

考题分布统计

年份	题型	主要考点
2025	递推方程×2 + 伪代码分析 + 贪心/分治 + LP + DP/贪心/回溯综合 + 平摊分析	递推、平摊、综合设计
2023	递推方程×2 + 贪心(面试) + ILP + 回溯(分派) + 平摊(最小队列) + DP(硬盘快递) + 分治(网格极小值)	主定理、贪心证明、平摊势能法
2022	递推方程 + 逆序对 + 找坏硬币 + LP + MST唯一性 + 完美匹配 + 广播DP	分治设计、LP建模、DP

高频考点（必备）

1. 递推方程：每年必考，2—3 题，10 分。主定理三种情况要非常熟练。
2. 动态规划：几乎每年必考大题（15—20 分）。要求写出完整的5要素。
3. 贪心：每年必考（10—15 分），必须给出正确性证明。
4. 平摊分析：近年频繁出现（2023、2025 均有），势能法是重点。
5. 回溯/分支限界：大题（15分），要写代价函数。
6. LP：每年都有（10分），建模+求解（图解法）。

十、答题模板

递推方程答题模板

1. 写出 a 、 b 、 $f(n)$ 的值
2. 计算 $n^{\log_b a}$
3. 判断 $f(n)$ 与 $n^{\log_b a}$ 的关系，对应 Case 1/2/3
4. 若 Case 3 还需验证正则条件 $af(n/b) \leq cf(n)$
5. 给出结论 $T(n) = \Theta(\dots)$

DP 答题模板

1. **算法思想**：用 $dp[\dots]$ 表示...的最优值
2. **递推方程**： $dp[\dots] = \min / \max\{\dots\}$
3. **边界条件**： $dp[\dots] = \dots$
4. **标记函数**： $bt[\dots] = \dots$ ，记录最优选择
5. **求解顺序**：自底向上，按...顺序计算
6. **构造解**：根据 bt 数组反向追踪
7. **时间复杂度**： $O(\dots)$

贪心答题模板

1. **算法思想**：按...排序，每次选择...
2. **伪代码**：...
3. **正确性证明（交换论证）**：假设最优解 OPT 与贪心解 G 不同，设在第 k 步两者不同...交换 OPT 中第 k 步的选择为 G 的选择，证明目标函数值不变差...故 G 是最优解。
4. **时间复杂度**： $O(n \log n)$ （排序主导）

平摊分析答题模板（势能法）

1. **定义势能函数**： $\Phi(D) = \dots$
2. **初始势**： $\Phi(D_0) = 0$
3. **对每类操作计算平摊代价**：操作 X ：实际代价 $c_i = \dots$ ，势差 $\Phi(D_i) - \Phi(D_{i-1}) = \dots$ ，平摊代价 $\hat{a}_i = \dots = O(1)$
4. **结论**： $\sum c_i \leq \sum \hat{a}_i + \Phi(D_0) - \Phi(D_n) \leq O(n)$ ，故每操作平摊代价 $O(1)$

十一、易错点提醒

1. **主定理 Case 3**：不仅要检查 $f(n) = \Omega(n^{\log_b a + \epsilon})$ ，还要验证正则条件 $af(n/b) \leq cf(n)$ 。
2. $T(n) = 2T(n/2) + n \log n$ ：不能用主定理，必须用递归树，结果是 $O(n \log^2 n)$ 。

3. **动态规划**: 忘记写标记函数和解的构造会被扣分。
4. **贪心**: 只写策略不写证明会被大量扣分 (正确性证明占 50%)。
5. **平摊分析**: 势函数必须满足 $\Phi(D_n) \geq \Phi(D_0)$ (一般取 $\Phi(D_0) = 0$, 保证 $\Phi \geq 0$ 即可)。
6. **回溯代价函数**: 必须是下界 (乐观估计), 不能高估。
7. **LCS**: $C[i, j] = C[i - 1, j - 1] + 1$ 仅当 $x_i = y_j$ 时, 注意下标从 1 开始。
8. **最大子段和**: $C[0] = 0$ (空段), $OPT = \max_{0 \leq i \leq n} C[i]$ 包含 $C[0] = 0$ 处理全负的情况。

十二、快速记忆卡片

Master Theorem:

a = 子问题数, b = 规模缩小比, $f(n)$ = 额外代价。比较 $f(n)$ vs $n^{\log_b a}$:

- f 小 $\rightarrow n^{\log_b a}$ (Case 1)
- f 等 $\rightarrow n^{\log_b a} \cdot \log n$ (Case 2)
- f 大 $\rightarrow f(n)$ (Case 3, 需验证正则)

分治精华:

Karatsuba: 3 次乘法 $\rightarrow O(n^{1.585})$ | Strassen: 7 次乘法 $\rightarrow O(n^{2.807})$ | 最近点对: 条带内 ≤ 6 个比较 $\rightarrow O(n \log n)$

DP 五要素: 思想 $\rightarrow$ 递推方程 $\rightarrow$ 边界 $\rightarrow$ 标记 $\rightarrow$ 自底向上

贪心证明:

活动选择: 交换论证 (最早完成) | 最优装载: 归纳法 (最轻优先) | 延迟调度: 交换论证 (最早截止)

平摊三法:

聚集: T/n | 记账: 存信用 (≥ 0 恒成立) | 势能: $\Phi \geq 0$, $\hat{a}_i = c_i + \Delta\Phi$

核心势函数:

栈 Φ = 栈高度, PUSH $\rightarrow 2$, POP $\rightarrow 0$

计数器 Φ = 1 的个数, INCREMENT $\rightarrow 2$

动态表 $\Phi = 2 \cdot \text{num} - \text{size}$, INSERT $\rightarrow 3$

祝考试顺利!